

Contents

- 1 Quicksort
 - Introduction
 - Partitioning
 - Quicksort algorithm
 - Quicksort analysis
 - Randomized Quicksort
 - Conclusion

Quicksort

- Proposed by C. A. R. Hoare in 1962.
- Divide and Conquer algorithm – like merge sort.
- In-place algorithm – like insertion and heap sorts.
- Runs very well with tuning.
- Worst-case is $O(n^2)$, but for all practical purposes runs in $O(n \lg n)$.

Why do we want to study Quicksort?

One of the most widely used, and extensively studied, sorting algorithms.

Divide and conquer

Quicksort an n -element array:

- 1 *Divide* Partition the array into subarrays around a *pivot* x such that the elements in lower subarray $\leq x \leq$ elements in the upper subarray.



- 2 *Conquer* Recursively sort the two subarrays.
- 3 *Combine* Trivial – just concatenate the lower subarray, *pivot*, and the upper subarray.

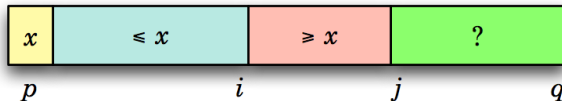
Partitioning algorithm

Algorithm

PARTITION(A, p, q) $\triangleright A[p..q]$

```
1   $x \leftarrow A[p]$             $\triangleright$  pivot =  $A[p]$ 
2   $i \leftarrow p$ 
3  for  $j \leftarrow p + 1$  to  $q$ 
4      do if  $A[j] \leq x$ 
5          then  $i \leftarrow i + 1$ 
6              exchange  $A[i] \leftrightarrow A[j]$ 
7  exchange  $A[p] \leftrightarrow A[i]$ 
8  return  $i$ 
```

Invariant



Quicksort algorithm

Algorithm

QUICKSORT(A, p, r) $\triangleright A[p..r]$

```
1  if  $p < r$ 
2      then  $q \leftarrow \text{PARTITION}(A, p, r)$ 
3          QUICKSORT( $A, p, q - 1$ )
4          QUICKSORT( $A, q + 1, r$ )
```

Initial call

QUICKSORT($A, 1, n$)

Analyzing Quicksort - worst-case performance

- Worst-case happens when pivot is always the minimum or maximum element.
- Result is that one of the partitions is always empty.
- When? **Input sorted (either non-decreasing or non-increasing)**

Worst-case analysis

(Note: the worst-case running time for partitioning is $\Theta(n)$.)

$$\begin{aligned}T(n) &= T(0) + T(n-1) + \Theta(n) \\&= \Theta(1) + T(n-1) + \Theta(n) \\&= T(n-1) + \Theta(n) \\&= \Theta(n^2)\end{aligned}$$

Best- and almost-worst case performances

- Best-case happens when pivot is the **median** element, creating equal size partitions.

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \lg n)$$

- What if the split is always $\frac{1}{10} : \frac{9}{10}$?

$$\begin{aligned} T(n) &= T(n/10) + T(9n/10) + \Theta(n) \\ &= \Theta(n \lg n) \quad \triangleright \text{See text for details} \end{aligned}$$

Key observation

Very close to worst-case produces $\Theta(n \lg n)$, not $\Theta(n^2)$.

How to ensure that we don't *usually* hit the worst-case?

Randomized Quicksort

- Pick a **random** pivot and partition around it.
- Pivot independent of input order, so no specific input produces worst-case behavior.
- The worst-case is determined only by the output of a random number generator.

RANDOMIZED-PARTITION(A, p, r) $\triangleright A[p..r]$

```
1   $i \leftarrow \text{RANDOM}(p, r)$   $\triangleright i = [p..r]$   
2  exchange  $A[p] \leftrightarrow A[i]$   
3  return PARTITION( $A, p, r$ )
```

RANDOMIZED-QUICKSORT(A, p, r)

```
1  if  $p < r$   
2      then  $q \leftarrow \text{RANDOMIZED-PARTITION}(A, p, r)$   
3          RANDOMIZED-QUICKSORT( $A, p, q - 1$ )  
4          RANDOMIZED-QUICKSORT( $A, q + 1, r$ )
```


Conclusion

- One of the most widely used sorting algorithm.
- While it runs in $O(n^2)$ time in the worst-case, it runs in $O(n \lg n)$ time on the average.
- Runs almost twice as fast as merge-sort.
- Can be *tuned* substantially.
- Almost all program language runtime library provide some variant of Quicksort (`java.util.Arrays.sort()` in Java, `qsort()` in C, `std::sort()` in C++, etc).

Questions to ask (and remember)

- What are the worst, best and average case performances?
- Is it in-place?
- Is it stable?